Fully Homomorphic Encryption over the Integers

Paper by van Dijk, Gentry, Halevi, and Vaikuntanathan (2010)

Nirav Bhattad (23B3307), Kunal Chaudhari (22B1060)

April 14, 2026

Outline

1. The FHE Dream and The Bootstrapping Blueprint
2. The Integer Engine and The Noise Crisis
3. Security Reduction and Known Attacks
4. Squashing the Decryption Circuit and Bootstrapping
5. Evolution and Open Problems

Motivation: The Cloud Computing Conundrum

Motivation

*We possess robust cryptographic solutions for data **in transit** (TLS/SSL) and data **at rest** (AES). However, computing on that data traditionally requires decryption, creating a massive vulnerability when outsourcing to untrusted cloud servers.*

Motivation: The Cloud Computing Conundrum

Motivation

*We possess robust cryptographic solutions for data **in transit** (TLS/SSL) and data **at rest** (AES). However, computing on that data traditionally requires decryption, creating a massive vulnerability when outsourcing to untrusted cloud servers.*

The Fully Homomorphic Encryption (FHE) Goal: Allow an untrusted party to take encrypted inputs, perform arbitrary, unbounded computations on them, and return an encrypted result.

Motivation: The Cloud Computing Conundrum

Motivation

*We possess robust cryptographic solutions for data **in transit** (TLS/SSL) and data **at rest** (AES). However, computing on that data traditionally requires decryption, creating a massive vulnerability when outsourcing to untrusted cloud servers.*

The Fully Homomorphic Encryption (FHE) Goal: Allow an untrusted party to take encrypted inputs, perform arbitrary, unbounded computations on them, and return an encrypted result.

When decrypted, this result must match exactly what the data owner would have obtained by computing on the plaintext locally.

The Quest for the Simplest Secure Scheme

"What is the simplest encryption scheme for which one can hope to achieve security?"

The Quest for the Simplest Secure Scheme

"What is the simplest encryption scheme for which one can hope to achieve security?"

The Caesar Cipher Analogy:

- Shift 'A' by 3 and 'B' by 3. You can trivially add those shifts together.
- It is perfectly (and ironically) homomorphic under addition.
- **Fatal Flaw:** It is completely insecure, because the ciphertexts are just shifted versions of the plaintexts, making them easily breakable by frequency analysis.

The Quest for the Simplest Secure Scheme

"What is the simplest encryption scheme for which one can hope to achieve security?"

The Caesar Cipher Analogy:

- Shift 'A' by 3 and 'B' by 3. You can trivially add those shifts together.
- It is perfectly (and ironically) homomorphic under addition.
- **Fatal Flaw:** It is completely insecure, because the ciphertexts are just shifted versions of the plaintexts, making them easily breakable by frequency analysis.

The Paradigm Shift (2009 vs. 2010):

- Craig Gentry's breakthrough 2009 FHE scheme utilized ideal lattices over polynomial rings – using conceptually and mathematically heavy machinery like Algebraic Number Theory and Algebraic Geometry.

The Quest for the Simplest Secure Scheme

"What is the simplest encryption scheme for which one can hope to achieve security?"

The Caesar Cipher Analogy:

- Shift 'A' by 3 and 'B' by 3. You can trivially add those shifts together.
- It is perfectly (and ironically) homomorphic under addition.
- **Fatal Flaw:** It is completely insecure, because the ciphertexts are just shifted versions of the plaintexts, making them easily breakable by frequency analysis.

The Paradigm Shift (2009 vs. 2010):

- Craig Gentry's breakthrough 2009 FHE scheme utilized ideal lattices over polynomial rings – using conceptually and mathematically heavy machinery like Algebraic Number Theory and Algebraic Geometry.
- **This Paper:** Asks if we can achieve the exact same FHE blueprint using only elementary addition and multiplication over the integers.

Formalizing the FHE Dream

To prove a scheme works, we must rigorously define it. A homomorphic public key encryption scheme $\mathcal{E}$ consists of four Probabilistic Polynomial-Time (PPT) algorithms:

Formalizing the FHE Dream

To prove a scheme works, we must rigorously define it. A homomorphic public key encryption scheme $\mathcal{E}$ consists of four Probabilistic Polynomial-Time (PPT) algorithms:

$$\mathcal{E} = (\text{KeyGen}, \text{Encrypt}, \text{Decrypt}, \textbf{Evaluate})$$

The Evaluate Algorithm: Takes as input a public key pk, a boolean circuit C, and a tuple of ciphertexts $\vec{c} = \langle c_1, ..., c_t \rangle$ (corresponding to the t input bits of C), and outputs a new evaluated ciphertext c_{out}.

Correctness and Homomorphism

Definition (Correct Homomorphic Decryption)

The scheme $\mathcal{E}$ is correct for a given t-input circuit C if, for any key-pair $(sk, pk) \leftarrow \mathsf{KeyGen}(\lambda)$, any plaintexts $m_1, ..., m_t \in \{0, 1\}$, and any ciphertexts $c_i \leftarrow \mathsf{Encrypt}(pk, m_i)$, the following holds with overwhelming probability:

$$\mathsf{Decrypt}(sk, \mathsf{Evaluate}(pk, C, \vec{c})) = C(m_1, ..., m_t)$$

Correctness and Homomorphism

Definition (Correct Homomorphic Decryption)

The scheme $\mathcal{E}$ is correct for a given t-input circuit C if, for any key-pair $(sk, pk) \leftarrow \text{KeyGen}(\lambda)$, any plaintexts $m_1, ..., m_t \in \{0, 1\}$, and any ciphertexts $c_i \leftarrow \text{Encrypt}(pk, m_i)$, the following holds with overwhelming probability:

$$\text{Decrypt}(sk, \text{Evaluate}(pk, C, \vec{c})) = C(m_1, ..., m_t)$$

Definition (Fully Homomorphic Encryption)

The scheme $\mathcal{E}$ is homomorphic for a class $\mathcal{C}$ of circuits if it is correct for all circuits $C \in \mathcal{C}$. The scheme $\mathcal{E}$ is **fully homomorphic** if it is correct for *all* boolean circuits (i.e., circuits of arbitrary depth and size).

The "Trivial" Solution and Circuit Privacy

Loophole: Can we trivially satisfy the definition of FHE right now?

The "Trivial" Solution and Circuit Privacy

Loophole: Can we trivially satisfy the definition of FHE right now?
Yes. An untrusted server could just take the circuit C, concatenate it to the raw input ciphertexts, and send it back. The user decrypts the inputs, reads C, and computes it locally.

The "Trivial" Solution and Circuit Privacy

Loophole: Can we trivially satisfy the definition of FHE right now?
Yes. An untrusted server could just take the circuit C, concatenate it to the raw input ciphertexts, and send it back. The user decrypts the inputs, reads C, and computes it locally.
To ban this useless scheme, cryptography enforces two strict properties. The first is **Circuit Privacy**.

The "Trivial" Solution and Circuit Privacy

Loophole: Can we trivially satisfy the definition of FHE right now?
Yes. An untrusted server could just take the circuit C, concatenate it to the raw input ciphertexts, and send it back. The user decrypts the inputs, reads C, and computes it locally.
To ban this useless scheme, cryptography enforces two strict properties. The first is **Circuit Privacy**.

Definition (Circuit Privacy)

The ciphertext output by the Evaluate algorithm must not reveal anything about the circuit C that was evaluated, beyond the final output value itself – even to the data owner who holds the secret key.

The "Trivial" Solution and Circuit Privacy

Loophole: Can we trivially satisfy the definition of FHE right now?
Yes. An untrusted server could just take the circuit C, concatenate it to the raw input ciphertexts, and send it back. The user decrypts the inputs, reads C, and computes it locally.
To ban this useless scheme, cryptography enforces two strict properties. The first is **Circuit Privacy**.

Definition (Circuit Privacy)

The ciphertext output by the Evaluate algorithm must not reveal anything about the circuit C that was evaluated, beyond the final output value itself – even to the data owner who holds the secret key.

The Compactness Constraint

The second (and far more difficult) property that rules out the trivial solution is **Compactness**.

The Compactness Constraint

The second (and far more difficult) property that rules out the trivial solution is **Compactness**.

The Goal: We must physically prevent the server from passing the computational burden back to the user. The server must be forced to actually do the math and compress the answer.

The Compactness Constraint

The second (and far more difficult) property that rules out the trivial solution is **Compactness**.

The Goal: We must physically prevent the server from passing the computational burden back to the user. The server must be forced to actually do the math and compress the answer.

Definition (Compactness)

A homomorphic scheme $\mathcal{E}$ is compact if there exists a fixed polynomial bound $b(\lambda)$ such that the size of the ciphertext output by Evaluate$(pk, C, \vec{c})$ is strictly bounded by $b(\lambda)$ bits, **independently of the size or depth of** C.

The Compactness Constraint

The second (and far more difficult) property that rules out the trivial solution is **Compactness**.

The Goal: We must physically prevent the server from passing the computational burden back to the user. The server must be forced to actually do the math and compress the answer.

Definition (Compactness)

A homomorphic scheme $\mathcal{E}$ is compact if there exists a fixed polynomial bound $b(\lambda)$ such that the size of the ciphertext output by Evaluate$(pk, C, \vec{c})$ is strictly bounded by $b(\lambda)$ bits, **independently of the size or depth of** C.

Intuition (The Root of the FHE Challenge)

Compactness is the true bottleneck of FHE. We cannot let the ciphertext grow proportionally to the circuit depth. We must compress the output so the user experiences $\mathcal{O}(1)$ decryption time, regardless of whether the cloud server evaluated a 10-gate circuit or a 10-billion-gate circuit. This strict bounding is exactly what forces us to manage noise, and ultimately, what forces us to use Bootstrapping.

The Crisis of Noise

Intuition (The Noise Bottleneck)

In all known FHE schemes, ciphertexts contain "noise" to ensure security.

- *Homomorphic addition adds the noise.*
- *Homomorphic multiplication* ***multiplies*** *the noise.*

Because multiplication squares the noise, evaluating a deep circuit causes the noise to explode past the decryption boundary, permanently destroying the plaintext.

The Crisis of Noise

Intuition (The Noise Bottleneck)

In all known FHE schemes, ciphertexts contain "noise" to ensure security.

- *Homomorphic addition adds the noise.*
- *Homomorphic multiplication* ***multiplies*** *the noise.*

Because multiplication squares the noise, evaluating a deep circuit causes the noise to explode past the decryption boundary, permanently destroying the plaintext.

This limits us to a *Somewhat* Homomorphic Encryption (SHE) scheme. How do we compute infinitely if noise acts as a strict depth limit?

Defining Bootstrappability

The answer is Gentry's Bootstrapping Blueprint. First, we define a specific class of circuits.

Defining Bootstrappability

The answer is Gentry's Bootstrapping Blueprint. First, we define a specific class of circuits.

Definition (Augmented Decryption Circuits)

Let $\mathcal{E}$ be an encryption scheme where decryption is implemented by a boolean circuit. The set of augmented decryption circuits, $D_{\mathcal{E}}$, consists of two circuits:

1. Decrypt two ciphertexts and ADD the results.
2. Decrypt two ciphertexts and MULTIPLY the results.

Defining Bootstrappability

The answer is Gentry's Bootstrapping Blueprint. First, we define a specific class of circuits.

Definition (Augmented Decryption Circuits)

Let $\mathcal{E}$ be an encryption scheme where decryption is implemented by a boolean circuit. The set of augmented decryption circuits, $D_{\mathcal{E}}$, consists of two circuits:

1. Decrypt two ciphertexts and ADD the results.
2. Decrypt two ciphertexts and MULTIPLY the results.

Definition (Bootstrappable Encryption)

Let $\mathcal{C}_{\mathcal{E}}$ be the set of permitted circuits a scheme can evaluate before noise exceeds the bounds. A scheme is **bootstrappable** if it can evaluate its own augmented decryption circuits:

$$D_{\mathcal{E}} \subseteq \mathcal{C}_{\mathcal{E}}$$

Gentry's Bootstrapping Theorem

Theorem (Gentry's Transformation)

There is an efficient transformation that, given a description of a bootstrappable scheme $\mathcal{E}$, outputs a description of a ***Fully Homomorphic*** *encryption scheme capable of evaluating circuits of any depth.*

Gentry's Bootstrapping Theorem

Theorem (Gentry's Transformation)

There is an efficient transformation that, given a description of a bootstrappable scheme $\mathcal{E}$, outputs a description of a ***Fully Homomorphic*** *encryption scheme capable of evaluating circuits of any depth.*

Intuition (How Bootstrapping "Refreshes" Ciphertexts)

1. *Take a ciphertext c whose noise is dangerously high.*
2. *Encrypt c again under a new public key.*
3. *Provide the server with an encrypted secret key.*
4. *The server homomorphically evaluates the decryption algorithm on the inner ciphertext.*

Gentry's Bootstrapping Theorem

Theorem (Gentry's Transformation)

There is an efficient transformation that, given a description of a bootstrappable scheme $\mathcal{E}$, outputs a description of a ***Fully Homomorphic*** *encryption scheme capable of evaluating circuits of any depth.*

Intuition (How Bootstrapping "Refreshes" Ciphertexts)

1. *Take a ciphertext c whose noise is dangerously high.*
2. *Encrypt c again under a new public key.*
3. *Provide the server with an encrypted secret key.*
4. *The server homomorphically evaluates the decryption algorithm on the inner ciphertext.*

Result: *Because the decryption circuit is evaluated homomorphically, the output is a fresh ciphertext encrypting the exact same message, but with a fixed, smaller noise level determined solely by the depth of the decryption circuit itself!*

Building the Engine: The Symmetric "Toy" Scheme

Before we look at the complex public-key version, we start with a beautifully simple symmetric-key foundation.

Building the Engine: The Symmetric "Toy" Scheme

Before we look at the complex public-key version, we start with a beautifully simple symmetric-key foundation.

Parameters: Let η be the bit-length of the secret key, and ρ be the bit-length of the noise.

Building the Engine: The Symmetric "Toy" Scheme

Before we look at the complex public-key version, we start with a beautifully simple symmetric-key foundation.

Parameters: Let η be the bit-length of the secret key, and ρ be the bit-length of the noise.

The Symmetric Algorithms

- **KeyGen:** The secret key p is a randomly chosen odd integer $p \in [2^{\eta-1}, 2^{\eta})$.
- **Encrypt(p, m):** To encrypt a bit $m \in \{0, 1\}$, choose a large random integer q and a small random noise integer r (where $|r| < 2^{\rho}$).

$$c = p \cdot q + 2r + m$$

- **Decrypt(p, c):** Output $m' = (c \pmod{p}) \pmod{2}$.

Intuition: Why Symmetric Decryption Works

Let's break down the decryption equation: $m' = (c \pmod{p}) \pmod{2}$.

Intuition: Why Symmetric Decryption Works

Let's break down the decryption equation: $m' = (c \pmod{p}) \pmod{2}$.

1. Substitute c: $(p \cdot q + 2r + m \pmod{p})$

Intuition: Why Symmetric Decryption Works

Let's break down the decryption equation: $m' = (c \pmod{p}) \pmod{2}$.

1. Substitute c: $(p \cdot q + 2r + m \pmod{p})$
2. Because $p \cdot q$ is an exact multiple of p, it vanishes perfectly. We are left with $(2r + m \pmod{p})$.

Intuition: Why Symmetric Decryption Works

Let's break down the decryption equation: $m' = (c \pmod{p}) \pmod{2}$.

1. Substitute c: $(p \cdot q + 2r + m \pmod{p})$
2. Because $p \cdot q$ is an exact multiple of p, it vanishes perfectly. We are left with $(2r + m \pmod{p})$.
3. **The Critical Condition:** The modulo p operation only leaves $2r + m$ unchanged if its absolute value is strictly less than half the modulus. We strictly require:

$$|2r + m| < p/2$$

Intuition: Why Symmetric Decryption Works

Let's break down the decryption equation: $m' = (c \pmod{p}) \pmod{2}$.

1. Substitute c: $(p \cdot q + 2r + m \pmod{p})$
2. Because $p \cdot q$ is an exact multiple of p, it vanishes perfectly. We are left with $(2r + m \pmod{p})$.
3. **The Critical Condition:** The modulo p operation only leaves $2r + m$ unchanged if its absolute value is strictly less than half the modulus. We strictly require:
$$|2r + m| < p/2$$
4. Assuming the noise is small enough, we take $(2r + m) \pmod{2}$. The $2r$ is even, so it vanishes, leaving exactly m.

Intuition: Why Symmetric Decryption Works

Let's break down the decryption equation: $m' = (c \pmod{p}) \pmod{2}$.

1. Substitute c: $(p \cdot q + 2r + m \pmod{p})$
2. Because $p \cdot q$ is an exact multiple of p, it vanishes perfectly. We are left with $(2r + m \pmod{p})$.
3. **The Critical Condition:** The modulo p operation only leaves $2r + m$ unchanged if its absolute value is strictly less than half the modulus. We strictly require:
$$|2r + m| < p/2$$
4. Assuming the noise is small enough, we take $(2r + m) \pmod{2}$. The $2r$ is even, so it vanishes, leaving exactly m.

Motivation

To anyone without p, the ciphertext c just looks like a massive, random integer. But algebraically, it is a point "tethered" to a multiple of p by a short, noisy string $(2r + m)$.

Symmetric Decryption: A Visualizable Example

Let's put numbers to the theory.

Setup:

- Secret Key $p = 31$
- Message bit $m = 1$

Symmetric Decryption: A Visualizable Example

Let's put numbers to the theory.

Setup:

- Secret Key $p = 31$
- Message bit $m = 1$

Encrypt: Choose a random $q = 5$ and a small noise $r = 2$.

$$c = (31 \cdot 5) + 2(2) + 1 = 155 + 4 + 1 = 160$$

The ciphertext is 160.

Symmetric Decryption: A Visualizable Example

Let's put numbers to the theory.

Setup:

- Secret Key $p = 31$
- Message bit $m = 1$

Encrypt: Choose a random $q = 5$ and a small noise $r = 2$.

$$c = (31 \cdot 5) + 2(2) + 1 = 155 + 4 + 1 = 160$$

The ciphertext is 160.

Decrypt:

$$160 \pmod{31} = 5$$

$$5 \pmod{2} = 1$$

We successfully recovered $m = 1$. The noise $(2r + m = 5)$ was safely smaller than $p/2$ (15.5).

Transitioning to Public-Key

A true FHE scheme requires public-key infrastructure. We achieve this by publishing a massive list of "encryptions of zero."

Transitioning to Public-Key

A true FHE scheme requires public-key infrastructure. We achieve this by publishing a massive list of "encryptions of zero."

New Parameters (based on security parameter λ):

- γ: Bit-length of the integers in the public key.
- τ: Number of integers in the public key.
- ρ': Secondary noise parameter for encryption $\rho' = \rho + \omega(\log \lambda)$.

Transitioning to Public-Key

A true FHE scheme requires public-key infrastructure. We achieve this by publishing a massive list of "encryptions of zero."

New Parameters (based on security parameter λ):

- γ: Bit-length of the integers in the public key.
- τ: Number of integers in the public key.
- ρ': Secondary noise parameter for encryption $\rho' = \rho + \omega(\log \lambda)$.

The Noise Distribution $\mathcal{D}_{\gamma,\rho}(p)$

To generate an "encryption of zero", choose $q \leftarrow \mathbb{Z} \cap [0, 2^\gamma/p)$ and $r \leftarrow \mathbb{Z} \cap (-2^\rho, 2^\rho)$. Output $x = pq + r$.

The Public-Key Algorithms

KeyGen:

- Sample $\tau + 1$ integers $x_0, x_1, \ldots, x_\tau$ from $\mathcal{D}_{\gamma,\rho}(p)$.
- Relabel so x_0 is the largest. Restart unless x_0 is odd and its noise $r_p(x_0)$ is even.
- Public key $pk = \langle x_0, x_1, \ldots, x_\tau \rangle$.

The Public-Key Algorithms

KeyGen:

- Sample $\tau + 1$ integers $x_0, x_1, \ldots, x_\tau$ from $\mathcal{D}_{\gamma,\rho}(p)$.
- Relabel so x_0 is the largest. Restart unless x_0 is odd and its noise $r_p(x_0)$ is even.
- Public key $pk = \langle x_0, x_1, \ldots, x_\tau \rangle$.

Encrypt(pk, m): Choose a random subset $S \subseteq \{1, 2, \ldots, \tau\}$ and a noise integer $r \in (-2^{\rho'}, 2^{\rho'})$.

$$c = \left[m + 2r + 2 \sum_{i \in S} x_i \right]_{x_0}$$

The Public-Key Algorithms

KeyGen:

- Sample $\tau + 1$ integers $x_0, x_1, \ldots, x_\tau$ from $\mathcal{D}_{\gamma,\rho}(p)$.
- Relabel so x_0 is the largest. Restart unless x_0 is odd and its noise $r_p(x_0)$ is even.
- Public key $pk = \langle x_0, x_1, \ldots, x_\tau \rangle$.

Encrypt(pk, m): Choose a random subset $S \subseteq \{1, 2, \ldots, \tau\}$ and a noise integer $r \in (-2^{\rho'}, 2^{\rho'})$.

$$c = \left[m + 2r + 2 \sum_{i \in S} x_i \right]_{x_0}$$

Intuition

Every x_i is effectively 0 (plus noise). Summing a random subset of x_i computes $0 + 0 + 0 \ldots$, creating a fresh, randomized encryption of zero. We multiply by 2 to keep the noise even, add our own fresh noise $2r$, and add m. The final modulo x_0 prevents the physical bit-size from growing infinitely large.

The Structure of a Fresh Ciphertext

To prove this scheme is homomorphic, we first mathematically prove that a fresh public-key ciphertext has the exact same structure as our symmetric toy scheme.

The Structure of a Fresh Ciphertext

To prove this scheme is homomorphic, we first mathematically prove that a fresh public-key ciphertext has the exact same structure as our symmetric toy scheme.

Lemma

Let $c \leftarrow$ Encrypt(pk, m). Then, $c = a \cdot p + (2b + m)$ for some integers a and b where the noise is bounded by $|2b + m| \leq \tau 2^{\rho+2}$.

The Structure of a Fresh Ciphertext

To prove this scheme is homomorphic, we first mathematically prove that a fresh public-key ciphertext has the exact same structure as our symmetric toy scheme.

Lemma

Let $c \leftarrow$ Encrypt(pk, m). Then, $c = a \cdot p + (2b + m)$ for some integers a and b where the noise is bounded by $|2b + m| \leq \tau 2^{\rho+2}$.

Proof Sketch:

- By definition, $c = m + 2r + 2\sum_{i \in S} x_i + k \cdot x_0$.
- We know every $x_i = q_i p + r_i$. Substitute this in:

$$c = p \cdot \left(kq_0 + 2\sum_{i \in S} q_i \right) + \left(m + 2r + 2k \cdot r_0 + 2\sum_{i \in S} r_i \right)$$

- The first term is a multiple of p (our $a \cdot p$). The second term is our accumulated noise $(2b + m)$. Its parity is m, and its magnitude is bounded.

Homomorphic Evaluation: Addition

Let the Evaluate algorithm simply be integer addition and multiplication. Let $c_1 = q_1 p + 2r_1 + m_1$ and $c_2 = q_2 p + 2r_2 + m_2$.

Homomorphic Evaluation: Addition

Let the Evaluate algorithm simply be integer addition and multiplication. Let $c_1 = q_1 p + 2r_1 + m_1$ and $c_2 = q_2 p + 2r_2 + m_2$.

Homomorphic Addition:

$$c_1 + c_2 = p(q_1 + q_2) + 2(r_1 + r_2) + (m_1 + m_2)$$

Homomorphic Evaluation: Addition

Let the Evaluate algorithm simply be integer addition and multiplication. Let $c_1 = q_1 p + 2r_1 + m_1$ and $c_2 = q_2 p + 2r_2 + m_2$.

Homomorphic Addition:

$$c_1 + c_2 = p(q_1 + q_2) + 2(r_1 + r_2) + (m_1 + m_2)$$

Observation: The structural format $a \cdot p +$ noise is perfectly preserved! We have a new multiple of p, plus $2 \times$ (new noise), plus the sum of the messages.

Crucially: The noise grows **linearly** $(r_1 + r_2)$.

Homomorphic Evaluation: Multiplication

Now, let us multiply the ciphertexts:

$$c_1 \cdot c_2 = (q_1 p + 2r_1 + m_1)(q_2 p + 2r_2 + m_2)$$

Homomorphic Evaluation: Multiplication

Now, let us multiply the ciphertexts:

$$c_1 \cdot c_2 = (q_1 p + 2r_1 + m_1)(q_2 p + 2r_2 + m_2)$$

Expanding this algebraically yields:

$$c_1 \cdot c_2 = p\big(p \cdot q_1 q_2 + q_1(2r_2 + m_2) + q_2(2r_1 + m_1)\big)$$

$$+ 2\big(2r_1 r_2 + r_1 m_2 + r_2 m_1\big) + m_1 m_2$$

Homomorphic Evaluation: Multiplication

Now, let us multiply the ciphertexts:

$$c_1 \cdot c_2 = (q_1 p + 2r_1 + m_1)(q_2 p + 2r_2 + m_2)$$

Expanding this algebraically yields:

$$c_1 \cdot c_2 = p\big(p \cdot q_1 q_2 + q_1(2r_2 + m_2) + q_2(2r_1 + m_1)\big)$$

$$+ 2\big(2r_1 r_2 + r_1 m_2 + r_2 m_1\big) + m_1 m_2$$

Observation: The structure is still preserved. We have a massive multiple of p (first term), plus $2 \times$ (mixed noise), plus the product of the messages $m_1 m_2$.

The Crisis: Look at the dominant noise term: $\mathbf{2r_1 r_2}$. When we multiply ciphertexts, the noise does not add – it **multiplies**.

The Decryption Constraint & The Noise Crisis

Constraint: Recall our strict rule for correct decryption. The modulo p operation $(c \pmod{p})$ will completely destroy the plaintext unless the absolute value of the total accumulated noise is strictly less than $p/2$.

The Decryption Constraint & The Noise Crisis

Constraint: Recall our strict rule for correct decryption. The modulo p operation $(c \pmod{p})$ will completely destroy the plaintext unless the absolute value of the total accumulated noise is strictly less than $p/2$.

The Crisis of the Square

Because multiplication squares the magnitude of the noise, the noise bit-length doubles with every sequential multiplication. It grows exponentially with the multiplicative depth of the circuit. We will rapidly hit the $p/2$ boundary.

Formalizing the Bottleneck

Exactly how deep can our circuit be before the noise destroys the data?

Formalizing the Bottleneck

Exactly how deep can our circuit be before the noise destroys the data?

Lemma (Permitted Polynomials)

Let $f(x_1, \ldots, x_t)$ be the multivariate polynomial computed by a circuit, and let d be its degree. The scheme can correctly evaluate this polynomial as long as:

$$d \leq \frac{\eta - 4 - \log ||\vec{f}||_1}{\rho' + 2}$$

where $||\vec{f}||_1$ is the l_1 norm of the coefficient vector of f.

Formalizing the Bottleneck

Exactly how deep can our circuit be before the noise destroys the data?

Lemma (Permitted Polynomials)

Let $f(x_1, \ldots, x_t)$ be the multivariate polynomial computed by a circuit, and let d be its degree. The scheme can correctly evaluate this polynomial as long as:

$$d \leq \frac{\eta - 4 - \log ||\vec{f}||_1}{\rho' + 2}$$

where $||\vec{f}||_1$ is the l_1 norm of the coefficient vector of f.

Intuition: The Noise Budget

- Think of η (the bit-size of p) as our total **Noise Budget**.
- Think of ρ' (the initial noise size) as the **cost** of one multiplication.
- Therefore, the maximum multiplicative depth (degree d) we can handle is roughly our total budget divided by the cost per operation: $\approx \eta/\rho'$.

Story so far

Where do we stand?

Story so far

Where do we stand?

1. We have successfully built an encryption scheme that can evaluate additions and multiplications without the server knowing the underlying data.

Story so far

Where do we stand?

1. We have successfully built an encryption scheme that can evaluate additions and multiplications without the server knowing the underlying data.
2. However, because of the multiplicative noise bottleneck, this scheme is strictly a **Somewhat Homomorphic Encryption (SHE)** scheme.

Story so far

Where do we stand?

1. We have successfully built an encryption scheme that can evaluate additions and multiplications without the server knowing the underlying data.
2. However, because of the multiplicative noise bottleneck, this scheme is strictly a **Somewhat Homomorphic Encryption (SHE)** scheme.
3. If we evaluate a circuit deeper than $d \approx \eta/\rho'$, the noise wraps around the modulus p, and the data is lost forever.

Story so far

Where do we stand?

1. We have successfully built an encryption scheme that can evaluate additions and multiplications without the server knowing the underlying data.
2. However, because of the multiplicative noise bottleneck, this scheme is strictly a **Somewhat Homomorphic Encryption (SHE)** scheme.
3. If we evaluate a circuit deeper than $d \approx \eta/\rho'$, the noise wraps around the modulus p, and the data is lost forever.

To achieve our dream of infinite FHE computation, we must find a way to evaluate our own decryption circuit within this strict noise budget. This leads us to the next part - the reduction of the hardness of the problem to a known hard problem, and the squashing of the decryption circuit.

The Need for Provable Security

Motivation

We have established a working mathematical engine, but in modern cryptography, we do not simply guess that a scheme is secure. We must show that breaking the semantic security of our encryption is mathematically equivalent to solving a known, well-studied hard problem.

The Need for Provable Security

Motivation

We have established a working mathematical engine, but in modern cryptography, we do not simply guess that a scheme is secure. We must show that breaking the semantic security of our encryption is mathematically equivalent to solving a known, well-studied hard problem.

For RSA, that problem is integer factorization. For Gentry's 2009 FHE, it is finding short vectors in ideal lattices.

The Need for Provable Security

Motivation

We have established a working mathematical engine, but in modern cryptography, we do not simply guess that a scheme is secure. We must show that breaking the semantic security of our encryption is mathematically equivalent to solving a known, well-studied hard problem.

For RSA, that problem is integer factorization. For Gentry's 2009 FHE, it is finding short vectors in ideal lattices.

For our integer-based scheme, the foundation is the **Approximate-GCD Problem**.

Defining Approximate-GCD

Recall our noise distribution $\mathcal{D}_{\gamma,\rho}(p)$, which outputs integers of the form $x = pq + r$, where x is roughly γ bits long, and the noise r is bounded by 2^ρ.

Defining Approximate-GCD

Recall our noise distribution $\mathcal{D}_{\gamma,\rho}(p)$, which outputs integers of the form $x = pq + r$, where x is roughly γ bits long, and the noise r is bounded by 2^ρ.

Definition (Approximate Integer Greatest Common Divisor)

The (ρ, η, γ)-approximate-gcd problem is: given polynomially many samples $x_0, x_1, \ldots, x_\tau$ drawn from $\mathcal{D}_{\gamma,\rho}(p)$ for a randomly chosen η-bit odd integer p, output the hidden integer p.

Defining Approximate-GCD

Recall our noise distribution $\mathcal{D}_{\gamma,\rho}(p)$, which outputs integers of the form $x = pq + r$, where x is roughly γ bits long, and the noise r is bounded by 2^ρ.

Definition (Approximate Integer Greatest Common Divisor)

The (ρ, η, γ)-approximate-gcd problem is: given polynomially many samples $x_0, x_1, \ldots, x_\tau$ drawn from $\mathcal{D}_{\gamma,\rho}(p)$ for a randomly chosen η-bit odd integer p, output the hidden integer p.

Intuition

If the noise $r = 0$, the problem is trivial. You are given exact multiples of p, and a single run of the 2,000-year-old Euclidean algorithm computes $\gcd(x_1, x_2) = p$ instantly. The addition of the noise term r is the sole cryptographic barrier.

The Search-to-Decision Reduction

Motivation

We must mathematically prove that breaking our scheme is equivalent to solving the Approximate-GCD problem. We do this via a "Search-to-Decision" reduction.

The Search-to-Decision Reduction

Motivation

We must mathematically prove that breaking our scheme is equivalent to solving the Approximate-GCD problem. We do this via a "Search-to-Decision" reduction.

Theorem

Any attack $\mathcal{A}$ with advantage ϵ on the semantic security of the encryption scheme can be converted into an algorithm $\mathcal{B}$ for solving the (ρ, η, γ)-approximate-gcd problem with success probability at least $\epsilon/2$.

The Search-to-Decision Reduction

Motivation

We must mathematically prove that breaking our scheme is equivalent to solving the Approximate-GCD problem. We do this via a "Search-to-Decision" reduction.

Theorem

Any attack $\mathcal{A}$ with advantage ϵ on the semantic security of the encryption scheme can be converted into an algorithm $\mathcal{B}$ for solving the (ρ, η, γ)-approximate-gcd problem with success probability at least $\epsilon/2$.

The Setup: Algorithm $\mathcal{B}$ is given random noisy multiples of a hidden integer p. Its goal is to find p. To do this, $\mathcal{B}$ uses the attacker $\mathcal{A}$ as a tool, feeding $\mathcal{A}$ simulated public keys and ciphertexts to extract information about p.

The Mechanism: Oracle and Binary GCD

How does $\mathcal{B}$ use the attacker $\mathcal{A}$ to find the secret key p?

The Mechanism: Oracle and Binary GCD

How does $\mathcal{B}$ use the attacker $\mathcal{A}$ to find the secret key p?

Step 1: The Learn-LSB Oracle

- $\mathcal{B}$ constructs randomized, simulated ciphertexts from a noisy integer z.
- By observing whether $\mathcal{A}$ correctly guesses the encrypted bits, $\mathcal{B}$ can deduce the **parity of the quotient** $q_p(z)$, without ever knowing p!

The Mechanism: Oracle and Binary GCD

How does $\mathcal{B}$ use the attacker $\mathcal{A}$ to find the secret key p?

Step 1: The Learn-LSB Oracle

- $\mathcal{B}$ constructs randomized, simulated ciphertexts from a noisy integer z.
- By observing whether $\mathcal{A}$ correctly guesses the encrypted bits, $\mathcal{B}$ can deduce the **parity of the quotient** $q_p(z)$, without ever knowing p!

Step 2: The Binary GCD Extraction

- $\mathcal{B}$ takes two noisy multiples of p and runs a standard Binary-GCD algorithm on them.
- Whenever the algorithm needs to know if a value is even or odd, $\mathcal{B}$ queries the Learn-LSB Oracle.
- **Result:** Iterating this process strips away the noise terms completely, reducing the values down to the exact secret key p.

Guaranteeing Success: The Distribution Match

For this reduction to hold, we must guarantee that attacker $\mathcal{A}$ cannot tell that it is being manipulated by $\mathcal{B}$. The simulated ciphertexts must look perfectly real.

Guaranteeing Success: The Distribution Match

For this reduction to hold, we must guarantee that attacker $\mathcal{A}$ cannot tell that it is being manipulated by $\mathcal{B}$. The simulated ciphertexts must look perfectly real.

Why $\mathcal{A}$ is successfully fooled:

1. **Quotient Uniformity:** By applying the **Leftover Hash Lemma**, the sum of the random subsets used to build the simulated ciphertext makes the underlying quotient statistically uniform.

Guaranteeing Success: The Distribution Match

For this reduction to hold, we must guarantee that attacker $\mathcal{A}$ cannot tell that it is being manipulated by $\mathcal{B}$. The simulated ciphertexts must look perfectly real.

Why $\mathcal{A}$ is successfully fooled:

1. **Quotient Uniformity:** By applying the **Leftover Hash Lemma**, the sum of the random subsets used to build the simulated ciphertext makes the underlying quotient statistically uniform.
2. **Noise Drowning:** $\mathcal{B}$ injects a massive random noise term (drawn from $2^{\rho'}$) into the simulation. This completely "drowns out" and statistically hides any underlying discrepancies in the base noise.

Guaranteeing Success: The Distribution Match

For this reduction to hold, we must guarantee that attacker $\mathcal{A}$ cannot tell that it is being manipulated by $\mathcal{B}$. The simulated ciphertexts must look perfectly real.

Why $\mathcal{A}$ is successfully fooled:

1. **Quotient Uniformity:** By applying the **Leftover Hash Lemma**, the sum of the random subsets used to build the simulated ciphertext makes the underlying quotient statistically uniform.
2. **Noise Drowning:** $\mathcal{B}$ injects a massive random noise term (drawn from $2^{\rho'}$) into the simulation. This completely "drowns out" and statistically hides any underlying discrepancies in the base noise.

Intuition

Because the simulated data is statistically indistinguishable from real FHE traffic, $\mathcal{A}$'s advantage holds. A rigorous counting argument proves that $\mathcal{B}$ will successfully recover the Approximate-GCD secret key p with probability $\geq \epsilon/2$.

Algebraic Failures: Continued Fractions

If we cannot break the semantic security, can we attack the Approximate-GCD directly?

Attempt 1: Continued Fractions Since $x_1 \approx q_1 p$ and $x_2 \approx q_2 p$, we know the public ratio x_1/x_2 is very close to the unknown rational q_1/q_2. Can we find q_1/q_2 in the continued fraction expansion of x_1/x_2?

Algebraic Failures: Continued Fractions

If we cannot break the semantic security, can we attack the Approximate-GCD directly?

Attempt 1: Continued Fractions Since $x_1 \approx q_1 p$ and $x_2 \approx q_2 p$, we know the public ratio x_1/x_2 is very close to the unknown rational q_1/q_2. Can we find q_1/q_2 in the continued fraction expansion of x_1/x_2?

The Mathematical Failure: Let's analyze the distance between the public ratio and the target quotient ratio:

$$\left|\frac{x_1}{x_2} - \frac{q_1}{q_2}\right| = \left|\frac{q_2(q_1 p + r_1) - q_1(q_2 p + r_2)}{q_2(q_2 p + r_2)}\right| \approx \left|\frac{q_2 r_1 - q_1 r_2}{p \cdot q_2^2}\right|$$

Algebraic Failures: Continued Fractions

If we cannot break the semantic security, can we attack the Approximate-GCD directly?

Attempt 1: Continued Fractions Since $x_1 \approx q_1 p$ and $x_2 \approx q_2 p$, we know the public ratio x_1/x_2 is very close to the unknown rational q_1/q_2. Can we find q_1/q_2 in the continued fraction expansion of x_1/x_2?

The Mathematical Failure: Let's analyze the distance between the public ratio and the target quotient ratio:

$$\left|\frac{x_1}{x_2} - \frac{q_1}{q_2}\right| = \left|\frac{q_2(q_1 p + r_1) - q_1(q_2 p + r_2)}{q_2(q_2 p + r_2)}\right| \approx \left|\frac{q_2 r_1 - q_1 r_2}{p \cdot q_2^2}\right|$$

The Cross-Noise Barrier

The distance is dominated by the cross-multiplied noise term $(q_2 r_1 - q_1 r_2)/p$. Because this distance is significantly larger than $1/q_2^2$, the target fraction is hopelessly obscured by the noise.

The Simultaneous Diophantine Approximation (SDA)

Because simple algebraic attacks fail, attackers must turn to the geometry of numbers – specifically lattice reduction algorithms like LLL.

The Simultaneous Diophantine Approximation (SDA)

Because simple algebraic attacks fail, attackers must turn to the geometry of numbers – specifically lattice reduction algorithms like LLL.

Given samples $x_0, x_1, \ldots, x_t$, we construct a $(t+1)$-dimensional SDA lattice L spanned by the rows of matrix M:

$$M = \begin{pmatrix} 2^\rho & x_1 & x_2 & \ldots & x_t \\ 0 & -x_0 & 0 & \ldots & 0 \\ 0 & 0 & -x_0 & \ldots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \ldots & -x_0 \end{pmatrix}$$

Finding the Target Vector

The attacker's goal is to find the hidden vector of quotients $\vec{q} = \langle q_0, q_1, \ldots, q_t \rangle$. Let's multiply $\vec{q}$ by M:

$$\vec{v} = \vec{q} \cdot M = \langle q_0 2^\rho,\ q_0 x_1 - q_1 x_0,\ \ldots,\ q_0 x_t - q_t x_0 \rangle$$

Finding the Target Vector

The attacker's goal is to find the hidden vector of quotients $\vec{q} = \langle q_0, q_1, \ldots, q_t \rangle$. Let's multiply $\vec{q}$ by M:

$$\vec{v} = \vec{q} \cdot M = \langle q_0 2^\rho,\ q_0 x_1 - q_1 x_0,\ \ldots,\ q_0 x_t - q_t x_0 \rangle$$

The Critical Cancellation: Look at the components $q_0 x_i - q_i x_0$. Since $x_i = q_i p + r_i$, we can expand:

$$\begin{aligned} q_0 x_i - q_i x_0 &= q_0(q_i p + r_i) - q_i(q_0 p + r_0) \\ &= q_0 q_i p + q_0 r_i - q_i q_0 p - q_i r_0 \\ &= \mathbf{q_0 r_i - q_i r_0} \end{aligned}$$

Finding the Target Vector

The attacker's goal is to find the hidden vector of quotients $\vec{q} = \langle q_0, q_1, \ldots, q_t \rangle$. Let's multiply $\vec{q}$ by M:

$$\vec{v} = \vec{q} \cdot M = \langle q_0 2^\rho,\ q_0 x_1 - q_1 x_0,\ \ldots,\ q_0 x_t - q_t x_0 \rangle$$

The Critical Cancellation: Look at the components $q_0 x_i - q_i x_0$. Since $x_i = q_i p + r_i$, we can expand:

$$\begin{aligned} q_0 x_i - q_i x_0 &= q_0(q_i p + r_i) - q_i(q_0 p + r_0) \\ &= q_0 q_i p + q_0 r_i - q_i q_0 p - q_i r_0 \\ &= \mathbf{q_0 r_i - q_i r_0} \end{aligned}$$

The Threat

The secret key p perfectly cancels out! This leaves $\vec{v}$ as a relatively short target vector in the lattice. If an algorithm like LLL can find $\vec{v}$, the attacker recovers the quotients, computes $p \approx x_0/q_0$, and the scheme is broken.

Defeating the Lattice

Will LLL actually find this target vector $\vec{v}$? This is where our parameter selection defends the scheme.

Defeating the Lattice

Will LLL actually find this target vector $\vec{v}$? This is where our parameter selection defends the scheme.

Scenario 1: Attacker uses a small number of samples t

- By Minkowski's bound, a low-dimensional lattice contains exponentially many *spurious* vectors that are much shorter than our target $\vec{v}$.
- The target vector is completely buried in geometric noise.

Defeating the Lattice

Will LLL actually find this target vector $\vec{v}$? This is where our parameter selection defends the scheme.

Scenario 1: Attacker uses a small number of samples t

- By Minkowski's bound, a low-dimensional lattice contains exponentially many *spurious* vectors that are much shorter than our target $\vec{v}$.
- The target vector is completely buried in geometric noise.

Scenario 2: Attacker uses a massive number of samples t

- The attacker forces t to be huge so $\vec{v}$ is the absolute shortest vector.
- However, known lattice reduction algorithms take time $\approx 2^{t/k}$ to find a 2^k approximation.
- Isolating $\vec{v}$ strictly requires time roughly $\mathbf{2}^{\gamma/\eta^2}$.

Summary of Secure Parameters

By setting the bit-length of the public key elements (γ) to be massively large, we force the lattice dimension to be computationally impossible to reduce.

Summary of Secure Parameters

By setting the bit-length of the public key elements (γ) to be massively large, we force the lattice dimension to be computationally impossible to reduce.

Final Secure Parameters:

- $\rho = \mathcal{O}(\lambda)$: Prevents brute-force guessing of the noise r.
- $\eta = \tilde{\mathcal{O}}(\lambda^2)$: Supports the homomorphic noise budget (depth for bootstrapping).
- $\gamma = \tilde{\mathcal{O}}(\lambda^5)$: Specifically ensures that $\gamma = \omega(\eta^2 \log \lambda)$. This makes the time $2^{\gamma/\eta^2}$ super-polynomial, thwarting all SDA and lattice reduction algorithms.

Summary of Secure Parameters

By setting the bit-length of the public key elements (γ) to be massively large, we force the lattice dimension to be computationally impossible to reduce.

Final Secure Parameters:

- $\rho = \mathcal{O}(\lambda)$: Prevents brute-force guessing of the noise r.
- $\eta = \tilde{\mathcal{O}}(\lambda^2)$: Supports the homomorphic noise budget (depth for bootstrapping).
- $\gamma = \tilde{\mathcal{O}}(\lambda^5)$: Specifically ensures that $\gamma = \omega(\eta^2 \log \lambda)$. This makes the time $2^{\gamma/\eta^2}$ super-polynomial, thwarting all SDA and lattice reduction algorithms.

Conclusion: While this results in massive public keys, it mathematically guarantees rigorous security based solely on the hardness of the Approximate-GCD over the integers.

Going from SHE to FHE: The Bootstrapping Blueprint

The Problem: Our Somewhat Homomorphic Encryption (SHE) scheme can add and multiply. However, every multiplication squares the noise.

Going from SHE to FHE: The Bootstrapping Blueprint

The Problem: Our Somewhat Homomorphic Encryption (SHE) scheme can add and multiply. However, every multiplication squares the noise.

Eventually the noise exceeds the decryption boundary $p/2$, and the ciphertext is permanently destroyed. How can we evaluate a circuit of arbitrary depth?

Going from SHE to FHE: The Bootstrapping Blueprint

The Problem: Our Somewhat Homomorphic Encryption (SHE) scheme can add and multiply. However, every multiplication squares the noise.

Eventually the noise exceeds the decryption boundary $p/2$, and the ciphertext is permanently destroyed. How can we evaluate a circuit of arbitrary depth?

The Solution: Bootstrapping (Gentry, 2009)
If a scheme can homomorphically evaluate its own decryption circuit plus one additional gate, it can evaluate *any* circuit of *any* depth.

Defining Bootstrappability

Definition (Augmented Decryption Circuits $\mathcal{D}_E(\lambda)$)

For a given security parameter λ, $\mathcal{D}_E(\lambda)$ consists of two circuits, both taking a secret key and two ciphertexts as input:

- One circuit decrypts both ciphertexts and **adds** the plaintext bits mod 2.
- One circuit decrypts both ciphertexts and **multiplies** the plaintext bits mod 2.

Definition (Bootstrappable Encryption)

A homomorphic scheme E is **bootstrappable** if, for every λ:

$$\mathcal{D}_E(\lambda) \subseteq \mathcal{C}_E(\lambda)$$

where $\mathcal{C}_E(\lambda)$ is the set of circuits E can correctly evaluate.

From Bootstrappability to Full Homomorphism

Theorem (Gentry 2009)

There is an efficient transformation that, given a bootstrappable scheme E and a parameter $d = d(\lambda)$, outputs a scheme $E^{(d)}$ such that:

1. $E^{(d)}$ *is compact, and*
2. $E^{(d)}$ *is correct for all circuits of depth up to* d.

Moreover, $E^{(d)}$ *is semantically secure if* E *is.*

Now we prove that our squashed scheme satisfies

$$\mathcal{D}_E \subset \mathcal{C}(\mathcal{P}_E)$$

so that the above theorem applies and we obtain a fully homomorphic scheme.

The Depth Problem: Why Naive Decryption Fails

Standard decryption:

$$m' \;=\; (c \bmod p) \bmod 2 \;=\; \big(c - p\lfloor c/p \rceil\big) \bmod 2$$

The Depth Problem: Why Naive Decryption Fails

Standard decryption:

$$m' \;=\; (c \bmod p) \bmod 2 \;=\; \big(c - p\lfloor c/p \rceil\big) \bmod 2$$

The roadblock: Computing $\lfloor c/p \rceil$ requires integer division. As a boolean circuit the carry chain is roughly λ AND-gate layers deep. After d AND layers, noise $\approx r^{2^d}$. With $d = \lambda$ this is catastrophic.

The Depth Problem: Why Naive Decryption Fails

Standard decryption:

$$m' \;=\; (c \bmod p) \bmod 2 \;=\; \big(c - p\lfloor c/p \rceil\big) \bmod 2$$

The roadblock: Computing $\lfloor c/p \rceil$ requires integer division. As a boolean circuit the carry chain is roughly λ AND-gate layers deep. After d AND layers, noise $\approx r^{2^d}$. With $d = \lambda$ this is catastrophic.

The permitted degree bound:
A circuit C belongs to $\mathcal{C}(\mathcal{P}_E)$ if its polynomial f satisfies:

$$d \;\leq\; \frac{\eta - 4 - \log \|\hat{f}\|_1}{\rho' + 2}$$

The division circuit's degree far exceeds this bound. We must **squash** the decryption circuit to bring it within reach.

Squashing: The Strategy

The key observation:
The division $\lfloor c/p \rceil$ is expensive because p is secret. If the key owner pre-computes a hint about $1/p$ and publishes it, the server can approximate the division using only public information – and the remaining homomorphic work becomes shallow.

Squashing: The Strategy

The key observation:
The division $\lfloor c/p \rceil$ is expensive because p is secret. If the key owner pre-computes a hint about $1/p$ and publishes it, the server can approximate the division using only public information – and the remaining homomorphic work becomes shallow.

Three new parameters:

- $\kappa = \gamma\eta/\rho'$: precision bits for the fixed-point approximation of $1/p$
- $\theta = \lambda$: Hamming weight of the sparse secret selector vector $\vec{s}$
- $\Theta = \omega(\kappa \cdot \log \lambda)$: total size of the public hint vector $\vec{y}$

Squashing: The Strategy

The key observation:
The division $\lfloor c/p \rceil$ is expensive because p is secret. If the key owner pre-computes a hint about $1/p$ and publishes it, the server can approximate the division using only public information – and the remaining homomorphic work becomes shallow.

Three new parameters:

- $\kappa = \gamma\eta/\rho'$: precision bits for the fixed-point approximation of $1/p$
- $\theta = \lambda$: Hamming weight of the sparse secret selector vector $\vec{s}$
- $\Theta = \omega(\kappa \cdot \log \lambda)$: total size of the public hint vector $\vec{y}$

The shift: Move the expensive $1/p$ computation from decrypt-time (constrained by the noise budget) to key-generation time (unconstrained, done once offline by the key owner).

Squashing: Modifying Key Generation

Starting from $\mathsf{sk}^* = p$ and pk^* of the base scheme:

Step 1 – Scaled approximation of $1/p$:

$$x_p \leftarrow \lfloor 2^\kappa / p \rceil$$

Computed by the key owner in the clear. After this, p is never needed again.

Squashing: Modifying Key Generation

Starting from $\mathsf{sk}^* = p$ and pk^* of the base scheme:

Step 1 – Scaled approximation of $1/p$:

$$x_p \leftarrow \lfloor 2^\kappa / p \rceil$$

Computed by the key owner in the clear. After this, p is never needed again.

Step 2 – Sparse secret selector $\vec{s}$:
Choose a random Θ-bit binary vector $\vec{s}$ with Hamming weight exactly θ. Set $S = \{i : s_i = 1\}$. $\vec{s}$ becomes the **new secret key**, replacing p.

Squashing: Modifying Key Generation

Starting from $\mathsf{sk}^* = p$ and pk^* of the base scheme:

Step 1 – Scaled approximation of $1/p$:

$$x_p \leftarrow \lfloor 2^\kappa / p \rceil$$

Computed by the key owner in the clear. After this, p is never needed again.

Step 2 – Sparse secret selector $\vec{s}$:
Choose a random Θ-bit binary vector $\vec{s}$ with Hamming weight exactly θ. Set $S = \{i : s_i = 1\}$. $\vec{s}$ becomes the **new secret key**, replacing p.

Step 3 – Public hint vector $\vec{y}$:
Choose random $u_i \in [0, 2^{\kappa+1})$ with $\sum_{i \in S} u_i \equiv x_p \pmod{2^{\kappa+1}}$. Set $y_i = u_i / 2^\kappa$. Publish $\vec{y} = \{y_1, \ldots, y_\Theta\}$. Each $y_i \in [0, 2)$ with κ bits of precision, satisfying:

$$\Big[\sum_{i \in S} y_i\Big]_2 = \frac{1}{p} - \Delta_p, \qquad |\Delta_p| < 2^{-\kappa}$$

Output: $\mathsf{sk} = \vec{s}, \quad \mathsf{pk} = (\mathsf{pk}^*,\ y_1, \ldots, y_\Theta)$.

Squashing: Modified Encrypt and Decrypt

Modified Encrypt:
Generate c^* as before. Then for each $i \in \{1, \ldots, \Theta\}$:

$$z_i \leftarrow [c^* \cdot y_i]_2$$

keeping only $n = \lceil \log \theta \rceil + 3$ bits of precision. Output $(c^*, z_1, \ldots, z_\Theta)$.

Squashing: Modified Encrypt and Decrypt

Modified Encrypt:
Generate c^* as before. Then for each $i \in \{1, \ldots, \Theta\}$:

$$z_i \leftarrow [c^* \cdot y_i]_2$$

keeping only $n = \lceil \log \theta \rceil + 3$ bits of precision. Output $(c^*,\ z_1, \ldots, z_\Theta)$.

Why the z_i encode the division:

$$\sum_{i \in S} z_i \approx c^* \sum_{i \in S} y_i \approx \frac{c^*}{p} \pmod 2$$

So $\lfloor \sum_i s_i z_i \rceil$ approximates $\lfloor c^*/p \rceil$ without the server ever knowing p or $\vec{s}$.

Squashing: Modified Encrypt and Decrypt

Modified Encrypt:
Generate c^* as before. Then for each $i \in \{1, \ldots, \Theta\}$:

$$z_i \leftarrow [c^* \cdot y_i]_2$$

keeping only $n = \lceil \log \theta \rceil + 3$ bits of precision. Output $(c^*, z_1, \ldots, z_\Theta)$.

Why the z_i encode the division:

$$\sum_{i \in S} z_i \approx c^* \sum_{i \in S} y_i \approx \frac{c^*}{p} \pmod 2$$

So $\lfloor \sum_i s_i z_i \rceil$ approximates $\lfloor c^*/p \rceil$ without the server ever knowing p or $\vec{s}$.

Modified Decrypt:

$$m' \leftarrow \left[c^* - \left\lfloor \sum_{i=1}^{\Theta} s_i z_i \right\rceil \right]_2$$

A sparse dot product of θ nonzero terms – far shallower than integer division.

The Complete FHE Scheme: KeyGen

$\textsc{KeyGen}(\lambda)$:

1. $p \leftarrow$ random odd η-bit integer (secret; used here only)
2. $\mathsf{pk}^* \leftarrow \{x_0, \ldots, x_\tau\}$ where $x_i = q_i p + 2r_i$ (encryptions of zero)
3. $x_p \leftarrow \lfloor 2^\kappa / p \rceil$
4. $\vec{s} \leftarrow$ random Θ-bit binary vector with $\mathrm{wt}(\vec{s}) = \theta$; $S = \{i : s_i = 1\}$
5. Choose u_i at random subject to $\sum_{i \in S} u_i \equiv x_p \pmod{2^{\kappa+1}}$; set $y_i = u_i / 2^\kappa$
6. Output $\mathsf{sk} = \vec{s}$ and $\mathsf{pk} = \left(\mathsf{pk}^*,\ y_1, \ldots, y_\Theta\right)$

The Complete FHE Scheme: Encrypt and Evaluate

Encrypt(pk**,** $m \in \{0,1\}$**):**

1. Choose random $T \subseteq \{1,\ldots,\tau\}$ and $r \in (-2^{\rho'}, 2^{\rho'})$
2. $c^* \leftarrow \left(m + 2r + 2\sum_{i\in T} x_i\right) \bmod x_0$
3. For each i: $z_i \leftarrow [c^* \cdot y_i]_2$, keeping $n = \lceil \log \theta \rceil + 3$ bits of precision
4. Output $(c^*,\ z_1,\ldots,z_\Theta)$

Evaluate(pk**,** C**,** $c_1,\ldots,c_t$**):**

1. Apply the integer addition and multiplication gates of C to the c^*-parts of each ciphertext
2. Noise grows additively under addition, quadratically under multiplication
3. Recompute $z_i \leftarrow [c^*_{\text{out}} \cdot y_i]_2$ for the output ciphertext

The Complete FHE Scheme: Decrypt

Decrypt(sk, $(c^*, \vec{z})$**):**

1. Sparse dot product: $\sigma \leftarrow \sum_{i=1}^{\Theta} s_i \cdot z_i$ (only θ nonzero terms)
2. Round: $v \leftarrow \lfloor \sigma \rceil$
3. Output $m' = (c^* - v) \bmod 2$

Why decryption is correct:
σ is within $\frac{1}{4}$ of the integer $\lfloor c^*/p \rceil$, so rounding always recovers the correct value.
The bound $\frac{1}{4}$ comes from two error terms, each at most $\frac{1}{16}$:

- Truncation error from n-bit precision of z_i: $|\sum_i s_i \Delta_i| \leq \frac{1}{16}$
- Approximation error from $x_p \approx 2^\kappa/p$: $|c^* \cdot \Delta_p| < \frac{1}{16}$

Total error $< \frac{1}{8} < \frac{1}{2}$, so rounding is unambiguous.

The Scheme is Bootstrappable

Theorem

Let E be the squashed scheme and $\mathcal{D}_E$ its augmented decryption circuits. Then $\mathcal{D}_E \subset \mathcal{C}(\mathcal{P}_E)$. That is, E is bootstrappable, and by [Gentry09, Theorem 4], E is fully homomorphic.

Security: Two Hardness Assumptions

The squashed scheme requires two independent hardness assumptions.

Security: Two Hardness Assumptions

The squashed scheme requires two independent hardness assumptions.

Assumption 1 – Approximate GCD (AGCD):
Given many samples $x_i = q_i \cdot p + r_i$ where r_i is small and p is a hidden large odd integer, find p.

Defends the base scheme: public key elements and fresh ciphertexts are computationally indistinguishable from random integers.

Security: Two Hardness Assumptions

The squashed scheme requires two independent hardness assumptions.

Assumption 1 – Approximate GCD (AGCD):
Given many samples $x_i = q_i \cdot p + r_i$ where r_i is small and p is a hidden large odd integer, find p.

Defends the base scheme: public key elements and fresh ciphertexts are computationally indistinguishable from random integers.

Assumption 2 – Sparse Subset Sum (SSSP):
Given $y_1, \ldots, y_\Theta$ and the knowledge that some θ-sparse subset sums to $\approx 1/p \pmod 2$, find the subset S.

Defends the hint vector $\vec{y}$: knowing $\vec{y}$ reveals nothing about which θ indices form S.

Security: Two Hardness Assumptions

The squashed scheme requires two independent hardness assumptions.

Assumption 1 – Approximate GCD (AGCD):
Given many samples $x_i = q_i \cdot p + r_i$ where r_i is small and p is a hidden large odd integer, find p.

Defends the base scheme: public key elements and fresh ciphertexts are computationally indistinguishable from random integers.

Assumption 2 – Sparse Subset Sum (SSSP):
Given $y_1, \ldots, y_\Theta$ and the knowledge that some θ-sparse subset sums to $\approx 1/p \pmod 2$, find the subset S.

Defends the hint vector $\vec{y}$: knowing $\vec{y}$ reveals nothing about which θ indices form S.

Note: SSSP is a separate assumption, not implied by AGCD.

The Evolution of Integer FHE

The original 2010 DGHV scheme was a conceptual breakthrough, but suffered from massive public key sizes ($\tilde{\mathcal{O}}(\lambda^{10})$) and severe performance bottlenecks. Subsequent research rapidly optimized the integer paradigm:

The Evolution of Integer FHE

The original 2010 DGHV scheme was a conceptual breakthrough, but suffered from massive public key sizes ($\tilde{\mathcal{O}}(\lambda^{10})$) and severe performance bottlenecks. Subsequent research rapidly optimized the integer paradigm:

1. **Key Compression & Modulus Switching (2011–2012):**
   - Evaluated quadratic forms and adopted "modulus switching" (adapted from the BGV scheme) to slash the public key size down to $\tilde{\mathcal{O}}(\lambda^5)$.
   - *Crucial impact:* Completely eliminated the need for the complex "squashing" step prior to bootstrapping.

The Evolution of Integer FHE

The original 2010 DGHV scheme was a conceptual breakthrough, but suffered from massive public key sizes ($\tilde{\mathcal{O}}(\lambda^{10})$) and severe performance bottlenecks. Subsequent research rapidly optimized the integer paradigm:

1. **Key Compression & Modulus Switching (2011–2012):**
   - Evaluated quadratic forms and adopted "modulus switching" (adapted from the BGV scheme) to slash the public key size down to $\tilde{\mathcal{O}}(\lambda^5)$.
   - *Crucial impact:* Completely eliminated the need for the complex "squashing" step prior to bootstrapping.
2. **Batching & SIMD Operations (2013):**
   - Utilized the Chinese Remainder Theorem (CRT) with multiple secret primes to encrypt vectors of plaintext bits into a single ciphertext, drastically improving amortized efficiency.

The Evolution of Integer FHE

The original 2010 DGHV scheme was a conceptual breakthrough, but suffered from massive public key sizes ($\tilde{\mathcal{O}}(\lambda^{10})$) and severe performance bottlenecks. Subsequent research rapidly optimized the integer paradigm:

1. **Key Compression & Modulus Switching (2011–2012):**
   - Evaluated quadratic forms and adopted "modulus switching" (adapted from the BGV scheme) to slash the public key size down to $\tilde{\mathcal{O}}(\lambda^5)$.
   - *Crucial impact:* Completely eliminated the need for the complex "squashing" step prior to bootstrapping.
2. **Batching & SIMD Operations (2013):**
   - Utilized the Chinese Remainder Theorem (CRT) with multiple secret primes to encrypt vectors of plaintext bits into a single ciphertext, drastically improving amortized efficiency.
3. **Scale-Invariant Noise & Message Spaces (2014–2015):**
   - Achieved linear (rather than exponential) noise growth, dropping the modulus switching requirement entirely.
   - Expanded the message space from binary ($\mathbb{Z}_2$) to any prime ($\mathbb{Z}_p$), reducing the multiplicative degree of the decryption circuit.

Open Problems and the Future of FHE

While the integer-based approach has been heavily optimized, several critical challenges remain for the Fully Homomorphic Encryption community at large:

Open Problems and the Future of FHE

While the integer-based approach has been heavily optimized, several critical challenges remain for the Fully Homomorphic Encryption community at large:

1. **The Bootstrapping Bottleneck:**
   - Even with scale-invariant noise and SIMD batching, homomorphically evaluating the decryption circuit remains a massive computational tax. Minimizing or entirely bypassing the need for bootstrapping remains the holy grail of FHE.

Open Problems and the Future of FHE

While the integer-based approach has been heavily optimized, several critical challenges remain for the Fully Homomorphic Encryption community at large:

1. **The Bootstrapping Bottleneck:**
   - Even with scale-invariant noise and SIMD batching, homomorphically evaluating the decryption circuit remains a massive computational tax. Minimizing or entirely bypassing the need for bootstrapping remains the holy grail of FHE.
2. **Ciphertext Expansion Ratio:**
   - The ratio of the ciphertext size to the actual underlying plaintext data is still highly impractical for low-bandwidth environments or massive databases.

Open Problems and the Future of FHE

While the integer-based approach has been heavily optimized, several critical challenges remain for the Fully Homomorphic Encryption community at large:

1. **The Bootstrapping Bottleneck:**
   - Even with scale-invariant noise and SIMD batching, homomorphically evaluating the decryption circuit remains a massive computational tax. Minimizing or entirely bypassing the need for bootstrapping remains the holy grail of FHE.
2. **Ciphertext Expansion Ratio:**
   - The ratio of the ciphertext size to the actual underlying plaintext data is still highly impractical for low-bandwidth environments or massive databases.
3. **RLWE vs. Approximate-GCD:**
   - Modern practical FHE libraries (like HElib, SEAL, and TFHE) have largely standardized on Ring Learning With Errors (RLWE) over polynomial rings.
   - *Open Question:* Can the conceptual simplicity of the integer approach (Approximate-GCD) ever be engineered to competitively match the hardware-accelerated speeds of RLWE schemes?

Conclusion

Thank You!

Are there any questions?